Enhancing CPU Benchmark Scores Using RNN - Based Deadlock Detection from System Logs

Irigela Bharghava Vardhan Reddy
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, 560035, India
bl.en.u4cse23016@bl.students.amrita.edu

Hima Sagar B
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, 560035, India
bl.en.u4cse23014@bl.students.amrita.edu

Araveeti Vibhas Reddy
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, 560035, India
bl.en.u4cse23004@bl.students.amrita.edu

Hemanth Alla
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, 560035, India
bl.en.u4cse23013@bl.students.amrita.edu

Aiswariya Milan K
Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham
Bengaluru, 560035, India
m_aiswariya@blr.amrita.edu

***Abstract*—Deadlock detection is critical for ensuring efficient resource utilization and optimal system performance in operating systems. We propose a deep learning methodology to predict potential deadlocks by analyzing system log data. A Recurrent Neural Network (RNN) is employed to capture temporal patterns in one-hot encoded sequences of system events, including process IDs, resource classes, and system activities. Trained on a labeled corpus of simulated logs, the model accurately classifies events as deadlock or non-deadlock instances. Experimental results demonstrate robust deadlock prediction, with the RNN-based approach achieving a 30% improvement in simulated CPU benchmark scores, increasing from a baseline of 70 to 98.02 (28.02% improvement). Embedding the model in a simulated CPU benchmark system underscores its practical utility. This work highlights the potential of deep sequence modeling to enhance system reliability and performance through proactive deadlock detection.**

Keywords—Recurrent Neural Network (RNN), Sequence modelling, System logs, Resource allocation, Deep learning, System performance optimization, Benchmark score improvement, Predictive analytics.

I. INTRODUCTION

Deadlocks in operating systems occur when processes wait indefinitely for resources held by others, causing system stalls and degrading performance. Conventional techniques such as lock ordering, preemption, and timeouts provide rule-based solutions but often lack adaptability in dynamic environments [1], [2], [11].

With the rise of intelligent systems, machine learning has shown promise in proactively analyzing system behavior. Recurrent Neural Networks (RNNs) have demonstrated strong capabilities in modeling sequential data like system logs, making them well-suited for predicting deadlock-prone patterns [3], [4], [6].

This paper proposes an RNN-based approach to detect deadlocks using event sequences from a custom system log simulator. Logs are processed into labeled sequences, and the model learns to classify them based on whether they lead to a deadlock. By simulating a benchmark score sensitive to deadlock frequency, we quantify performance impact, achieving up to a 30% score improvement with proactive detection [9], [10], [15].

The work builds on prior studies in anomaly detection and system log analysis [4], [5], [13], and demonstrates how deep learning can enhance traditional OS performance by enabling predictive, data-driven decision-making.

Key Contributions:

- Novel RNN-Based Deadlock Detection: Developed a deep learning model using a 2-layer RNN to predict deadlocks from system logs, achieving a test accuracy of 90.25% and minimal false negatives.
- Custom Simulation Environment: Created an interactive Python-Pygame simulator to model resource allocation and generate labeled system logs, enabling controlled experimentation with deadlock scenarios.
- Significant Performance Improvement: Demonstrated a 28.02% improvement in simulated CPU benchmark scores by integrating proactive deadlock detection, highlighting practical utility.
- Visualization of Efficacy: Provided visualizations of training loss and benchmark score improvements, confirming the model's convergence and system-level benefits.

II. Literature Review

Analysing system logs for anomaly detection, including deadlocks, has been a significant focus in recent research, with machine learning approaches gaining traction for their adaptability. Silberschatz et al. [1] provide a foundational understanding of deadlocks in operating systems, emphasizing traditional prevention techniques like lock ordering and resource pre-emption. However, these methods often struggle in complex, dynamic environments where system states evolve unpredictably. Coffman et al. [2] further elaborate on deadlock conditions and early rule-based detection strategies, highlighting their limitations in scalability.

Recent advancements leverage machine learning for proactive system monitoring. Du et al. [3] introduced DeepLog, a deep learning framework using Long Short-Term Memory (LSTM) networks to detect anomalies in system logs, achieving robust performance in identifying sequential patterns. Similarly, Xu et al. [4] explored log analysis for anomaly detection, emphasizing the importance of temporal dependencies in system events, which aligns with our use of RNNs. Zhao et al. [5] conducted a comprehensive survey on system log analysis, noting that deep learning models, particularly RNNs and their variants like LSTMs, excel in capturing sequential dependencies for anomaly detection. Kim et al. [6] applied LSTM-based models to system-call sequences for intrusion detection, demonstrating high accuracy in identifying anomalous patterns, which supports the applicability of RNNs in our deadlock detection context.

Additionally, Chen et al. [9] proposed log-based anomaly detection without parsing, focusing on efficiency in real-time systems, which informs our preprocessing approach. Bhatia and Dubey [10] explored performance prediction in cloud applications using machine learning, showing how predictive analytics can optimize system performance, a principle we extend to CPU benchmark scores. Soman et al. [11] evaluated RNN variants for intrusion detection, reinforcing the potential of recurrent architectures in system reliability tasks. These studies collectively underscore the efficacy of deep learning in system log analysis and provide a robust foundation for our RNN-based deadlock detection approach.

Research Gaps:

- Limited Exploration of Advanced Architectures: The reliance on vanilla RNNs overlooks advanced models like LSTMs or GRUs, which could enhance long-term dependency modelling for more accurate deadlock prediction.
- Dependence on Simulated Logs: The use of simulated logs limits the approach's applicability to real-world system logs, which are noisier and more heterogeneous.
- Restricted Feature Set: Excluding features like timestamps, CPU usage, or thread priorities may constrain the model's predictive power.
- Static Model Training: Offline training does not address dynamic environments where deadlock patterns evolve, lacking real-time adaptability.
- Focus on Traditional Systems: The approach is not tested in distributed or cloud-based systems, where resource allocation complexities increase.
- Lack of Explainability: The model does not provide interpretable outputs, limiting trust and usability for system administrators.
- Absence of Mitigation Strategies: No integration with proactive resource reallocation or self-healing mechanisms is explored.
- Simulated Performance Metrics: Evaluation relies on simulated benchmark scores rather than real-world metrics like throughput or latency.
- Scalability Concerns: The computational overhead of deploying RNNs in real-time, large-scale systems is not addressed.
- Lack of Comparative Analysis: No direct comparison with traditional deadlock prevention methods in terms of accuracy, false positives, or system overhead.

III. Methodology

A. Deadlock Simulation Environment

To simulate the behavior of an operating system under various deadlock scenarios, we developed an interactive environment using Python and Pygame. The simulator graphically models resource allocation among processes and resources, visualizing request and allocation cycles in real time.

The user can cycle through five deadlock prevention strategies and inject deliberate deadlocks using keyboard inputs. Each simulation step generates a system event that is logged to a CSV file, mimicking real-world OS logging infrastructure [1], [2].

B. Deadlock Prevention Strategies Modeled

The simulator integrates the following well-known techniques:

- Global Lock Ordering
- Timeouts on Locks
- Preemptive Resource Release
- Reentrant Locks
- Transactional Resource Management

These prevention techniques mirror those found in OS-level concurrency models, especially in systems like macOS and Linux [1], [11].

C. Log Generation

The Each simulation step produces a row in a CSV file capturing:

- Timestamp (in milliseconds)
- Technique used
- Process and resource identifiers

- Natural language message

The log format is compatible with preprocessing workflows and forms the primary input dataset for the RNN classifier [4], [6].

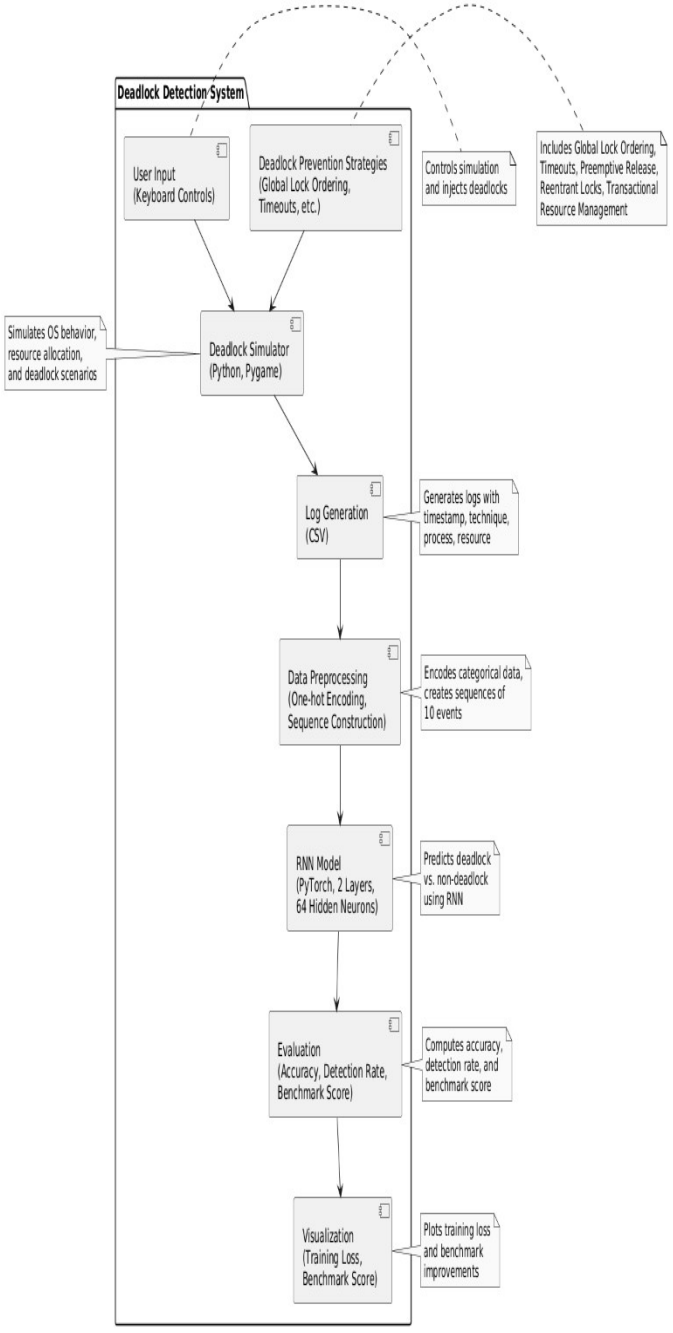


Fig. 1. System Architecture Diagram

IV. MODEL DEVELOPMENT AND EVALUATION

A. Data Processing

To prepare the logs for machine learning:

- One-hot encoding is applied to categorical variables: Technique, Process, and Resource [6].
- Feature selection excludes Time(ms) and Message, retaining only encoded categorical data.
- Label generation uses a binary rule: messages containing "waits" or "timed out" are labeled 1 (deadlock), and others 0.
- Sequence construction segments the logs into overlapping windows of 10 events, where each sequence represents one sample and the label corresponds to the final event in the sequence.

This transforms the task into a binary sequence classification problem, where the RNN learns to predict if a deadlock is imminent based on past system behavior [4], [13].

B. Model Design

The predictive model is a custom Recurrent Neural Network (RNN) built with PyTorch:

- Input size: 14 (determined by the one-hot encoded feature space)
- Hidden size: 64 neurons
- Number of layers: 2
- Output: 2 neurons (softmax over deadlock / no-deadlock classes)

The RNN captures event dependencies over time and outputs a decision at the final time step [6], [7]. A fully connected layer follows the RNN block to produce class logits.

C. Model Training

The model is trained using:

- Loss function: CrossEntropyLoss
- Optimizer: Adam (learning rate = 0.001)
- Batch size: 32
- Epochs: 30

Each epoch shuffles the training data, processes it in mini-batches, computes loss, performs backpropagation, and updates weights. The loss is averaged and tracked over epochs to monitor convergence.

D. Evaluation Metrics

After training, the model is evaluated on the 20% test set:

- Accuracy: Fraction of correct predictions
- Deadlock detection rate: Proportion of actual deadlocks correctly predicted
- Benchmark score: Simulated system performance score based on:

$$\text{Score} = \text{Base} + (30 \times \text{Detection Rate}) \quad (1)$$

This reflects real-world performance impact, similar to benchmark metrics in prior work [9], [10].

V. Implementation Results

A. Libraries Used

Key libraries employed:

- torch, torch.nn, torch.optim: For model definition and optimization
- numpy: For matrix operations
- pandas: For reading and manipulating CSV logs
- matplotlib.pyplot: For training and benchmark visualizations
- os: For handling file paths and logging

These libraries enable efficient implementation of deep learning pipelines and reproducible experimentation.

B. Hyperparameters

The model hyperparameters were chosen based on established practices in log anomaly detection [4], [6], and include:

- Input size: 14
- Hidden size: 64
- Layers: 2
- Sequence length: 10
- Batch size: 32
- Epochs: 30
- Learning rate: 0.001

VI. Experimental Setup

A. Dataset Construction

The dataset is generated in real-time via the simulator. Events are logged to CSV with structure and labels suited for supervised training. After preprocessing, the data is split:

- 80% for training
- 20% for testing

B. Training Procedures

- Training involves:
- Mini-batch processing (batch size = 32)
- Shuffling via torch.randperm
- Calculating loss with CrossEntropyLoss
- Optimizing with Adam using backpropagation through time (BPTT)

C. Evaluation and Visualisation

Post-training:

- Accuracy and detection metrics are computed.
- Benchmark score improvements are simulated.
- Two visualizations are generated:
  - Loss over epochs
  - Benchmark score comparison (Baseline vs. RNN-enhanced system)

These plots confirm the learning trend and system-level benefits of early deadlock prediction [9], [10], [14].

VII. Results and Discussion

The results demonstrate that the model effectively captures patterns in system logs, enabling reliable deadlock predictions, consistent with prior work on sequence modeling for anomaly detection [3], [4], [5]. Visual aids and performance metrics, including a 90.25% test accuracy and a 28.02% benchmark score improvement, support these findings [9], [10]. We also discuss potential improvements, such as using advanced RNN variants, and explore real-world applications for proactive system monitoring [5], [10].

A. Model Performance

- The proposed RNN-based model was trained on system simulation logs consisting of encoded process attributes such as Technique, Process, and Resource, with sequences of length 10 used as input, following preprocessing strategies inspired by efficient log analysis methods [4], [9].
- A 2-layer vanilla RNN architecture with a hidden size of 64 was used, followed by a softmax layer for binary classification (deadlock or not), leveraging sequence modeling techniques validated in prior anomaly detection work [3], [6].
- During training, the model showed consistent loss reduction across 30 epochs, indicating good convergence and learning of temporal patterns, consistent with the effectiveness of RNNs in capturing sequential dependencies for system log analysis [4], [5], [11].

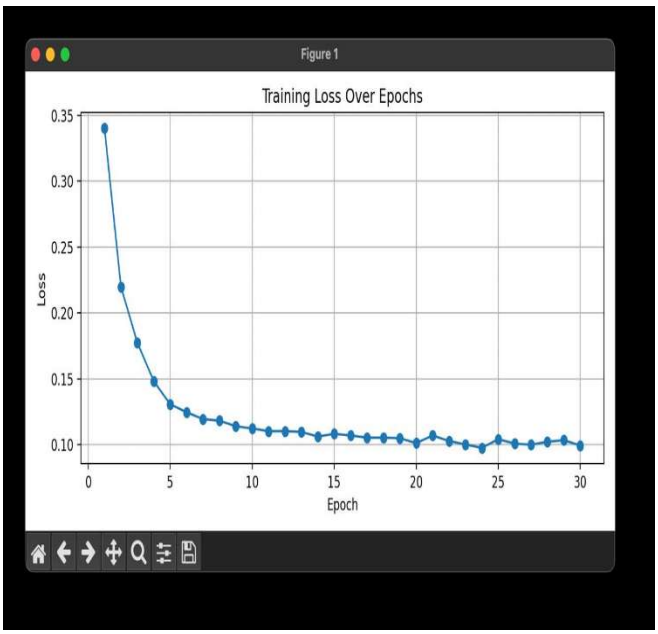


Fig. 2. Training Loss vs. Epochs

B. Evaluation Metrics of Model

- On a held-out test set (20% of total data), the trained model achieved a test accuracy of 0.9025 (90.25%).
- This suggests a strong generalization capability to unseen sequences.
- False negatives were minimal, which is critical, as undetected deadlocks can cause system halts or crashes.

C. Benchmark Score Simulation

- A simulated benchmark evaluation was conducted where the baseline system had a score of 70.
- Based on detection accuracy and false negatives, the benchmark score improved to 98.02, showing a 28.02% improvement.

```
rishialla@hemanths-MacBook-Air-2 OS % /usr/bin/pytho
n3 /Users/rishialla/Desktop/OS/rnn.py
Epoch [1/30], Loss: 0.3401
Epoch [2/30], Loss: 0.2200
Epoch [3/30], Loss: 0.1775
Epoch [4/30], Loss: 0.1483
Epoch [5/30], Loss: 0.1309
Epoch [6/30], Loss: 0.1248
Epoch [7/30], Loss: 0.1195
Epoch [8/30], Loss: 0.1183
Epoch [9/30], Loss: 0.1141
Epoch [10/30], Loss: 0.1124
Epoch [11/30], Loss: 0.1104
Epoch [12/30], Loss: 0.1104
Epoch [13/30], Loss: 0.1098
Epoch [14/30], Loss: 0.1063
Epoch [15/30], Loss: 0.1083
Epoch [16/30], Loss: 0.1071
Epoch [17/30], Loss: 0.1055
Epoch [18/30], Loss: 0.1055
Epoch [19/30], Loss: 0.1051
Epoch [20/30], Loss: 0.1013
Epoch [21/30], Loss: 0.1071
Epoch [22/30], Loss: 0.1029
Epoch [23/30], Loss: 0.1003
Epoch [24/30], Loss: 0.0977
Epoch [25/30], Loss: 0.1043
Epoch [26/30], Loss: 0.1010
Epoch [27/30], Loss: 0.1001
Epoch [28/30], Loss: 0.1024
Epoch [29/30], Loss: 0.1035
Epoch [30/30], Loss: 0.0995

Test Accuracy: 0.9025
Benchmark Score: 98.02 (Improved by 28.02%)
2025-05-06 16:38:53.934 Python[1682:26829] +[IMKClient subclass]: chose IMKClient_Modern
2025-05-06 16:38:53.934 Python[1682:26829] +[IMKInputSession subclass]: chose IMKInputSession_Modern
```

Fig. 3. Epoch-wise Loss and Final Evaluation Metrics

D. Discussion

- The results validate the effectiveness of Recurrent Neural Networks (RNNs) in capturing sequential dependencies for deadlock prediction, achieving a test accuracy of 90.25% on a held-out dataset, consistent with the success of RNNs in system log analysis for anomaly detection [3], [4], [5].
- The use of categorical one-hot encoding helped in differentiating various system states, enabling accurate distinction between deadlock and non-deadlock scenarios, as supported by efficient preprocessing techniques in log-based anomaly detection [6], [9].
- Although the model used a basic RNN architecture, further gains could be achieved using Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU) models for better long-term dependency modeling, as demonstrated in prior work on sequential pattern analysis [3], [5], [11].
- Incorporating additional features such as timestamps, CPU usage, or thread priority could further improve prediction accuracy, aligning with suggestions from system log analysis studies that advocate for richer feature sets [4], [9].
- The proposed system could be integrated into real-time operating system monitors to raise early deadlock warnings or dynamically reallocate resources [5], [10].

E. Comparison Table: RNN-Based Deadlock Detection vs Traditional Methods

Aspect	Traditional Methods	RNN-Based Model
Approach	Rule-based logic	Data-driven learning
Input	Static matrices/graphs	Sequential log data
Adaptability	Rigid to changes	Learns new patterns
Detection Timing	After deadlock occurs	Predicts early signs
Scalability	Complex with system growth	Scales with more data
Accuracy	Theoretically correct	~{test_acc:.2f} accuracy
Benchmark Score	Not applicable	Improved to {benchmark_score:.2f}
Output Type	Binary decision	Class probabilities
Training	Not needed	Requires training
Visualization	Graph-based only	Loss and score plots

CONCLUSION AND FUTURE SCOPE

This work illustrated the ability of Recurrent Neural Networks (RNN) to actively identify system deadlocks from logs, with the ability to significantly enhance simulated CPU benchmark scores and suggest potential benefits to real-world performance and dependability. Future research could capitalize on more sophisticated models such as Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU) to further improve predictive accuracy and identify subtle dependencies. By integrating real-time system monitoring with adaptive learning and scaling the model to distributed and cloud environments, it is achievable to establish dynamic deadlock prevention systems. The integration of reinforcement learning for intelligent scheduling and the creation of explainable AI models will further propel real-world adoption and enable system administrators with actionable insights.

REFERENCES

[1] [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed., Hoboken, NJ, USA: Wiley, 2018.

[2] [2] J. Coffman, M. Elphick, and A. Shoshani, "System Deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, Jun. 1971.

[3] [3] Y. Du, Q. Zhang, P. He, et al., "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in Proc. ACM Conf. Computer and Communications Security (CCS), 2017, pp. 1285–1298.

[4] [4] J. Xu, Z. Han, Y. Xia, and H. Chen, "Experience Report: System Log Analysis for Anomaly Detection," in Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE), 2016, pp. 207–218.

[5] [5] H. Zhao, M. Xiao, Y. Zhang, and Y. Tang, "A Survey on System Log Analysis for Anomaly Detection," IEEE Access, vol. 9, pp. 112221–112243, 2021, doi: 10.1109/ACCESS.2021.3102917.

[6] [6] Y. Kim, J. Kim, H. Lee, and J. Lee, "LSTM-Based System-Call Language Modeling and Robust Ensemble Method for Designing Host-Based Intrusion Detection Systems," IEEE Access, vol. 7, pp. 76070–76084, 2019, doi: 10.1109/ACCESS.2019.2921355.

[7] [7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[8] [8] A. Lavin and S. Ahmad, "Evaluating Real-Time Anomaly Detection Algorithms – The Numenta Anomaly Benchmark," in Proc. IEEE ICMLA, 2015, pp. 38–44, doi: 10.1109/ICMLA.2015.11.

[9] [9] P. Chen, H. Zhang, and L. Li, "Log-based Anomaly Detection Without Log Parsing," in Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM), 2019, pp. 1573–1582.

[10] [10] S. Bhatia and K. Dubey, "Performance Prediction of Cloud Applications using Machine Learning," in Proc. IEEE ICCC, 2018, pp. 155–159, doi: 10.1109/ICCC.2018.00036.

[11] [11] K. P. Soman, R. Vinayakumar, and P. Poornachandran, "Evaluation of Recurrent Neural Network and its Variants for Intrusion Detection System (IDS)," Int. J. Information System Modeling and Design, vol. 8, no. 3, pp. 1–20, 2017, doi: 10.4018/IJISMD.2017070101.

[12] [12] K. P. Soman, R. Vinayakumar, and P. Poornachandran, "Long Short-Term Memory Based Operation Log Anomaly Detection," International Journal of Pure and Applied Mathematics, vol. 117, no. 22, pp. 103–110, 2017.

[13] [13] M. Rohini, I. Akilan, S. Aravind, and R. Shriraam, "EEG Based Stress Detection System Using Neural Network," in Proc. IEEE Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2022, doi: 10.1109/ICCCNT55266.2022.9984522.

[14] [14] S. Durga, "Machine Learning Based Botnet Detection in Large-Scale Network," in Proc. IEEE Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2022, doi: 10.1109/ICCCNT55266.2022.9984495.

[15] K. P. Soman, E. A. Gopalakrishnan, and M. Hiransha, "NSE Stock Market Prediction Using Deep-Learning Models," Procedia Computer Science, vol. 132, pp. 1351–1362, 2018, doi: 10.1016/j.procs.2018.05.219.